



Universidad Nacional Autónoma de México

Facultad de Contaduría y Administración

Licenciatura en Informática

Nombre del alumno

Daniel Gerardo Reyes Castro

MATERIA

PROGRAMACION DE DISPOSITIVOS MOVILES

Nombre del Asesor

CRISTIAN CARDOSO ARELLANO

UNIDAD 5

Arreglos y tiempo de ejecución

ACTIVIDAD 1

Objetivo del aprendizaje

- Ejecutar una sentencia básica de programación en lenguaje Java

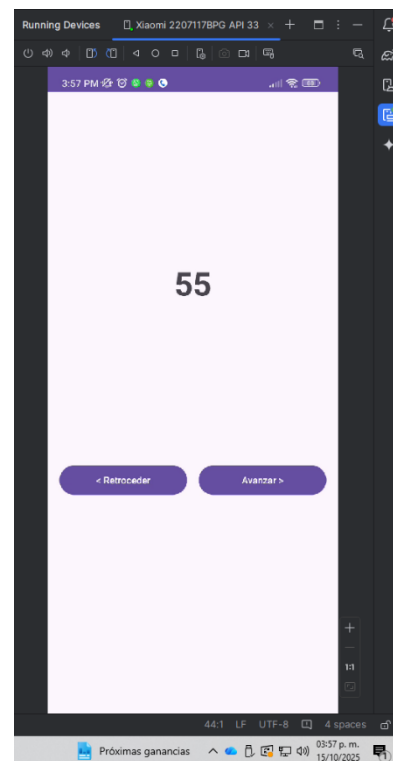
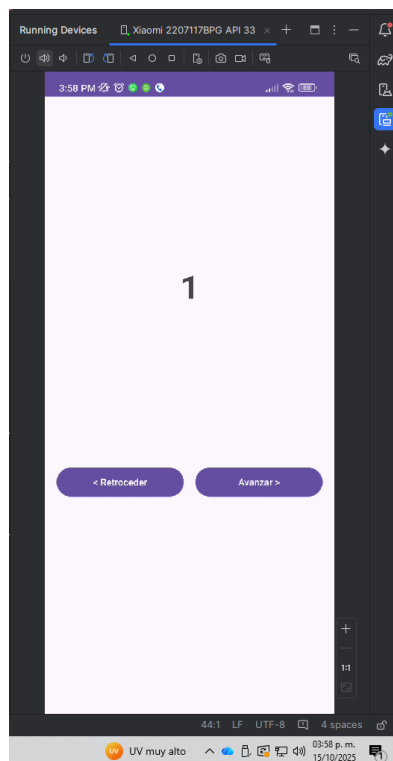
Actividad sugerida:

1. Crear tu primer activity
2. En lenguaje de programación JAVA dentro de Android Studio, crea una app vacía.
3. Crear un arreglo con 1 millón de elementos enteros de manera aleatoria.
4. Ordenar los elementos.
5. Medir el tiempo de inicio a fin.
6. Imprimir en el logcat los resultados obtenidos.

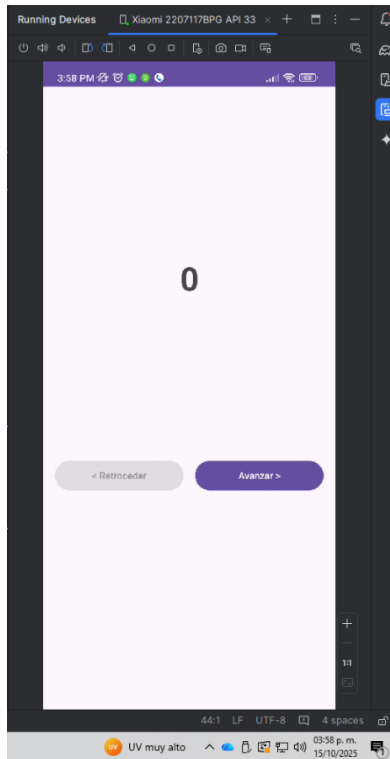
Subir actividad en el repositorio de github creado por el alumno.

1. android_2026_m1_03.pdf
2. código fuente.

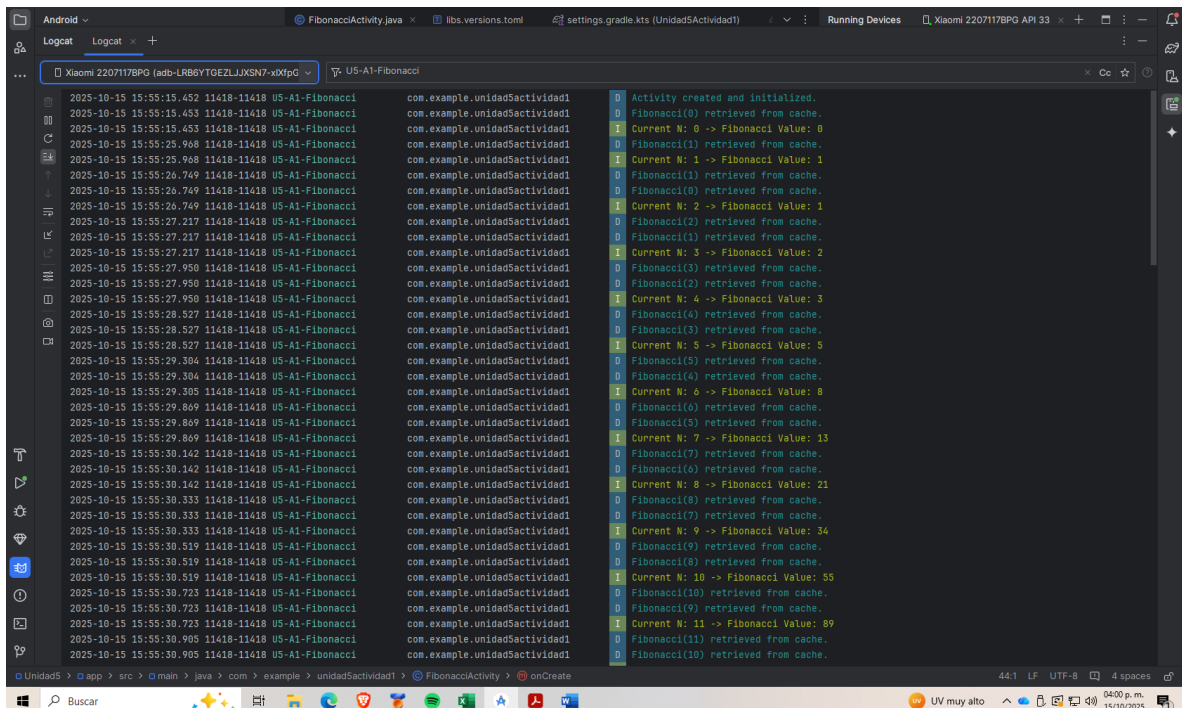
Interfaz de Usuario (UI) en Acción



Botón de Retroceder deshabilitado cuando el valor es 0 o 1 (demostrando la validación en el código).



Prueba de Memoization (Logcat)



Código fuente FibonacciActivity.java

```

package com.example.unidad5actividad1;

import androidx.appcompat.app.AppCompatActivity;

import android.os.Bundle;
import android.util.Log;
import android.widget.Button;
import android.widget.TextView;

import java.util.HashMap;
import java.util.Map;

public class FibonacciActivity extends AppCompatActivity {

    public static final String TAG = "U5-A1-Fibonacci";

    // View components
    private TextView resultadoTextView;
    private Button avanzarButton;
    private Button retrocederButton;

    // State variable for the Fibonacci counter ('n' value)
    private int contadorFibonacci = 0;

    // Cache (Memoization) to store already calculated results
    (optimization)
    private Map<Integer, Long> cache = new HashMap<>();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        // Ensure the correct layout file is loaded
        setContentView(R.layout.activity_fibonacci);
        Log.d(TAG, "Activity created and initialized.");

        // Initialize view components
        resultadoTextView = findViewById(R.id.textViewResultado);
        avanzarButton = findViewById(R.id.buttonAvanzar);
        retrocederButton = findViewById(R.id.buttonRetroceder);

        // Initialize cache with base cases: F(0)=0, F(1)=1
        cache.put(0, 0L);
        cache.put(1, 1L);

        // Restore state if the activity is recreated (e.g., screen
rotation)
        if (savedInstanceState != null) {
            contadorFibonacci = savedInstanceState.getInt("fib_count",
0);
        }

        // Configure Listeners for the buttons
        avanzarButton.setOnClickListener(v -> {
            contadorFibonacci++;
            actualizarVista();
        });
    }

```

```

        retrocederButton.setOnClickListener(v -> {
            // Only allow decrement if the counter is greater than 0
            if (contadorFibonacci > 0) {
                contadorFibonacci--;
                actualizarVista();
            }
        });

        // Initial view update
        actualizarVista();
    }

    // --- State Management ---
    @Override
    protected void onSaveInstanceState(Bundle outState) {
        super.onSaveInstanceState(outState);
        outState.putInt("fib_count", contadorFibonacci); // Save the
current 'n' value
        Log.d(TAG, "State saved: fib_count = " + contadorFibonacci);
    }

    // --- Fibonacci Logic with Memoization (Recursion) ---

    /**
     * Calculates the Fibonacci value for 'n' using recursion with
Memoization.
     * @param n The index of the sequence (e.g., n=6, value=8)
     * @return The Long Fibonacci value for F(n).
     */
    private long calcularFibonacci(int n) {
        if (n < 0) return 0;

        // 1. Memoization: If the value is already in the cache, return
it immediately
        if (cache.containsKey(n)) {
            Log.d(TAG, "Fibonacci(" + n + ") retrieved from cache.");
            return cache.get(n);
        }

        // 2. Recursion: Calculate the result using the recursive formula
        long resultado = calcularFibonacci(n - 1) + calcularFibonacci(n -
2);

        // 3. Memoization: Store the result in the cache before returning
        cache.put(n, resultado);

        return resultado;
    }

    /**
     * Updates the TextView with the calculated Fibonacci value and
controls button state.
     */
    private void actualizarVista() {
        long valorFib = calcularFibonacci(contadorFibonacci);

```

```

        // Log the current state
        Log.i(TAG, "Current N: " + contadorFibonacci + " -> Fibonacci
Value: " + valorFib);

        // Update the TextView
        resultadoTextView.setText(String.valueOf(valorFib));

        // Disable the back button when at the start (N=0)
        retrocederButton.setEnabled(contadorFibonacci > 0);
    }
}

```

Conclusion

Esta actividad nos ayuda a comprender los principios de la programación algorítmica. El desafío de implementar la secuencia de Fibonacci para un número 'n' se abordó mediante el uso de recursividad, lo que permitió una solución eficaz que refleja la naturaleza matemática de la serie.

Al emplear una estructura de datos HashMap (cache) para almacenar los valores de Fibonacci previamente calculados, se optimizó el rendimiento del algoritmo. Esta implementación evita el recálculo redundante, esencial para mantener la eficiencia del hilo principal de la UI, especialmente al trabajar con valores de 'n' elevados.

Fuentes de consulta

1. León Abreu, J, L. (s.f.) Fibonacci. Proyecto Arquímedes. https://arquimedes.matem.unam.mx/mati/actividades/actividad_fibonacci/
2. Java HashMap. (s.f.) W3Schools. https://www.w3schools.com/java/java_hashmap.asp
3. Jesús. (2024, abril 13) ¿Qué es un Hash Map? DONGEE. <https://www.dongee.com/tutoriales/que-es-un-hash-map/>
4. Feregrino, A. (2017, noviembre 27) Memoización. That C# guy. <https://thatcsharpguy.com/post/memoization>